

Fizetések létehozása

1. Fizetesek.jsx

A fizetések kezeléséhez elkészítettem a Fizetesek.jsx komponenst. A komponensben React state-ek segítségével tároltam a fizetések, a szerződések és az űrlap adatait. Gondoskodtam arról, hogy az oldal betöltésekor automatikusan lekérdeződjenek a szükséges adatok a backendből. Az űrlap segítségével lehetővé tettem új fizetések rögzítését és a meglévő fizetések módosítását.

```
import { useEffect, useState } from "react";
import "../styles/fizetesek.css";

function Fizetesek() {

  const [fizetesek, setFizetesek] = useState([]);
  const [szerzodesek, setSzerzodesek] = useState([]);
  const [szerkesztettId, setSzerkesztettId] = useState(null);
  const [statuszSzuro, setStatuszSzuro] = useState("Mind");

  const [ujFizetes, setUjFizetes] = useState({
    szerzodesid: "",
    osszeg: "",
    fizetesdatum: "",
    statusz: "Teljesítve",
    fizetesmod: "",
    megjegyzes: ""
  });

  const fizetesekBetoltese = async () => {

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/fizeteslistazas"
      );

      const adatok = await valasz.json();

      setFizetesek(adatok);

    } catch (err) {

      console.error(err);

    }

  };

}
```

```
const szerzodesekBetoltese = async () => {

  try {

    const valasz = await fetch(
      "http://localhost:3001/api/szerzodeslistazas"
    );

    const adatok = await valasz.json();

    setSzerzodesek(adatok);

  } catch (err) {

    console.error(err);

  }

};

useEffect(() => {

  fizetesekBetoltese();
  szerzodesekBetoltese();

}, []);

const mentes = async (e) => {

  e.preventDefault();

  try {

    let url =
      "http://localhost:3001/api/fizetesletrehozas";

    let method = "POST";

    if (szerkesztettId) {

      url =
        `http://localhost:3001/api/fizetesmodositas/${szerkesztettId}`;

      method = "PUT";

    }

    const valasz = await fetch(
```

```
        url,
        {
            method,
            headers: {
                "Content-Type":
                    "application/json"
            },
            body: JSON.stringify(
                ujFizetes
            )
        }
    );

    const adat =
        await valasz.json();

    alert(adat.uzenet);

    setUjFizetes({
        szerzodesid: "",
        osszeg: "",
        fizetesdatum: "",
        statusz: "Teljesítve",
        fizetesmod: "",
        megjegyzes: ""
    });

    setSzerkesztettId(null);

    fizetesekBetoltese();

} catch (err) {

    console.error(err);

    alert("Hiba történt!");

}

};

const torles = async (id) => {

    if (
        !window.confirm(
            "Biztosan törlöd a fizetést?"
        )
    ) {
```

```
        return;
    }

    try {

        const valasz = await fetch(
            `http://localhost:3001/api/fizetestorles/${id}`,
            {
                method: "DELETE"
            }
        );

        const adat =
            await valasz.json();

        alert(adat.uzenet);

        fizetesekBetoltese();

    } catch (err) {

        console.error(err);

    }

};

return (

    <div className="tenant-container">

        <h1>Fizetések</h1>

        <form
            className="tenant-form"
            onSubmit={mentes}
        >

            <select
                value={
                    ujFizetes.szerzodesid
                }
                onChange={(e) =>
                    setUjFizetes({
                        ...ujFizetes,
                        szerzodesid:
                            e.target.value
                    })
                }
            />

        </form>

    </div>

);
```

```
    }
    required
  >

  <option value="">
    Válassz szerződést
  </option>

  {szerzodesek.map(
    (szerzodes) => (

      <option
        key={
          szerzodes.id
        }
        value={
          szerzodes.id
        }
      >
        {
          szerzodes.berlonev
        }
      </option>

    )
  )}

</select>

<input
  type="number"
  placeholder="Összeg"
  value={
    ujFizetes.osszeg
  }
  onChange={ (e) =>
    setUjFizetes({
      ...ujFizetes,
      osszeg:
        e.target.value
    })
  }
  required
/>

<input
  type="date"
  value={
```

```
        ujFizetes.fizetesdatum
      }
      onChange={ (e) =>
        setUjFizetes({
          ...ujFizetes,
          fizetesdatum:
            e.target.value
        })
      }
      required
    />

    <select
      value={
        ujFizetes.statusz
      }
      onChange={ (e) =>
        setUjFizetes({
          ...ujFizetes,
          statusz:
            e.target.value
        })
      }
    >

      <option value="Teljesítve">
        Teljesítve
      </option>

      <option value="Függőben">
        Függőben
      </option>

      <option value="Sikertelen">
        Sikertelen
      </option>

    </select>

    <input
      type="text"
      placeholder="Fizetési mód"
      value={
        ujFizetes.fizetesmod
      }
      onChange={ (e) =>
        setUjFizetes({
          ...ujFizetes,
```

```
                fizetesmod:
                    e.target.value
            })
        }
        required
    />

    <input
        type="text"
        placeholder="Megjegyzés"
        value={
            ujFizetes.megjegyzes
        }
        onChange={ (e) =>
            setUjFizetes({
                ...ujFizetes,
                megjegyzes:
                    e.target.value
            })
        }
    />

    <button
        type="submit"
    >
        {
            szerkesztettId
                ? "Módosítás mentése"
                : "Fizetés rögzítése"
        }
    </button>

</form>

<div className="filter-container">

    <select
        value={statuszSzuro}
        onChange={ (e) =>
            setStatuszSzuro(
                e.target.value
            )
        }
    >

        <option value="Mind">
            Összes fizetés
        </option>
```

```
        <option value="Teljesítve">
            Teljesítve
        </option>

        <option value="Függőben">
            Függőben
        </option>

        <option value="Sikertelen">
            Sikertelen
        </option>

    </select>

</div>

<div className="tenant-grid">

    {fizetesek
        .filter((fizetes) => {

            if (
                statuszSzuro ===
                "Mind"
            ) {
                return true;
            }

            return (
                fizetes.statusz ===
                statuszSzuro
            );

        })
        .map(
            (fizetes) => (

                <div
                    className="tenant-card"
                    key={
                        fizetes.id
                    }
                >

                    <h3>
                        {
                            fizetes.berlonev
```



```
    }  
</h3>  
  
<p>  
    <strong>  
        Összeg:  
    </strong>{" "  
    {  
        fizetes.osszeg  
    } Ft  
</p>  
  
<p>  
    <strong>  
        Dátum:  
    </strong>{" "  
    {  
        new Date(  
            fizetes.fizetesdatum  
        ).toLocaleDateString(  
            "hu-HU"  
        )  
    }  
</p>  
  
<p>  
    <strong>  
        Fizetési mód:  
    </strong>{" "  
    {  
        fizetes.fizetesmod  
    }  
</p>  
  
{  
    fizetes.megjegyzes &&  
    (  
        <p>  
            <strong>  
                Megjegyzés:  
            </strong>{" "  
            {  
                fizetes.megjegyzes  
            }  
        </p>  
    )  
}
```

```
<p>

  <strong>
    Státusz:
  </strong>

  <span
    className={
      fizetes.statusz === "Teljesítve"
        ? "active-status"
        : fizetes.statusz === "Függőben"
        ? "pending-status"
        : "expired-status"
    }
  >
    {" "}
    {
      fizetes.statusz
    }
  </span>

</p>

<div className="buttons">

  <button
    type="button"
    className="edit-btn"
    onClick={() => {

      setSzerkesztettId(
        fizetes.id
      );

      setUjFizetes({
        szerzodesid:
          fizetes.szerzodesid,
        osszeg:
          fizetes.osszeg,
        fizetesdatum:
          fizetes.fizetesdatum?.split("T")[0] ||
          fizetes.fizetesdatum,
        statusz:
          fizetes.statusz,
        fizetesmod:
          fizetes.fizetesmod,
        megjegyzes:
          fizetes.megjegyzes || ""
      })
    }}
  >
    Szerkesztés
  </button>

  <button
    type="button"
    className="delete-btn"
    onClick={() => {
      deleteFizetes(fizetes.id)
    }}
  >
    Töröl
  </button>

</div>

</div>
</div>
```

```
        });

        window.scrollTo({
            top: 0,
            behavior: "smooth"
        });

    }}
    >
    Módosítás
</button>

<button
    type="button"
    className="delete-btn"
    onClick={() =>
        torles(
            fizetes.id
        )
    }
    >
    Törlés
</button>

</div>

</div>

    )
    }}

</div>

</div>

    );
}

export default Fizetesek;
```

2. fizeteseklistazasa.js

Megvalósítottam a fizetések listázását a backend API végpont használatával. A lekérdezett adatokat React állapotkezelés segítségével tároltam és jelenítettem meg. A fizetéseket kártyás elrendezésben jelenítettem meg a könnyebb áttekinthetőség érdekében. Minden kártyán

Szakmai Gyakorlat II. – Frontend – Backend integrációs szakasz

Orosz Kristóf – EYZWG9

feltüntettem a bérlő nevét, az összeget, a dátumot és a fizetés módját. A listázás során figyeltem arra, hogy az adatok naprakészen jelenjenek meg minden művelet után.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const fizetesek = await conn.query(`
      SELECT
        fizetesek.id,
        fizetesek.szerzodesid,
        fizetesek.osszeg,
        fizetesek.fizetesdatum,
        fizetesek.statusz,
        fizetesek.fizetesmod,
        fizetesek.megjegyzes,

        berlok.id AS berloid,
        berlok.nev AS berlonev,
        berlok.email AS berloemail,
        berlok.telefonszam,

        szerzodesek.ingatlanid,
        szerzodesek.kezdetdatum,
        szerzodesek.vegdatum,
        szerzodesek.havidij,
        szerzodesek.statusz AS szerzodesstatusz

      FROM fizetesek

      INNER JOIN szerzodesek
        ON szerzodesek.id = fizetesek.szerzodesid

      INNER JOIN berlok
        ON berlok.id = szerzodesek.berloid

      ORDER BY fizetesek.fizetesdatum DESC
    `);
```

```
        res.status(200).json(fizetesekek);

    } catch (err) {

        console.error("Fizetések lekérdezési hiba:", err);

        res.status(500).json({
            siker: false,
            uzenet: "Hiba történt a fizetések lekérdezése során!"
        });

    } finally {

        if (conn) {
            conn.release();
        }

    }

});

module.exports = router;
```

3. fizetesekletrehozasa.js

Kialakítottam egy adatbeviteli űrlapot új fizetések rögzítéséhez. Az űrlapon megadható a szerződés, az összeg, a fizetés dátuma, a státusz és a fizetési mód. Az adatok elküldése POST kérés segítségével történik a backend felé. A sikeres mentés után visszajelzést jelenítettem meg a felhasználó számára.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

    const {
        szerzodesid,
        osszeg,
        fizetesdatum,
        statusz,
        fizetesmod,
        megjegyzes
    } = req.body;

    let conn;
```

```
    try {

        conn = await pool.getConnection();

        await conn.query(
            `INSERT INTO fizetesek
            (szerzodesid, osszeg, fizetesdatum, statusz, fizetesmod, megjegyzes)
            VALUES (?, ?, ?, ?, ?, ?)`,
            [
                szerzodesid,
                osszeg,
                fizetesdatum,
                statusz,
                fizetesmod,
                megjegyzes
            ]
        );

        res.json({
            siker: true,
            uzenet: "Fizetes sikeresen rogzitve!"
        });

    } catch (err) {

        res.status(500).json(err);

    } finally {

        if(conn) conn.release();

    }

});

module.exports = router;
```

4. fizetesekmodositasa.js

Lehetővé tettem a korábban rögzített fizetések módosítását. A módosítás gombra kattintva a kiválasztott rekord adatait visszatöltöttem az űrlap mezőibe. A felhasználó ezután szerkesztheti az adatokat, majd elmentheti a változtatásokat. A módosított adatokat PUT kérés segítségével továbbítottam a backendnek. A mentés után a lista automatikusan frissül, így az új adatok azonnal láthatóvá válnak.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");
```

```
router.put("/:id", async (req, res) => {

  const {
    osszeg,
    fizetesdatum,
    statusz,
    fizetesmod,
    megjegyzes
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE fizetesek
      SET
      osszeg = ?,
      fizetesdatum = ?,
      statusz = ?,
      fizetesmod = ?,
      megjegyzes = ?
      WHERE id = ?`,
      [
        osszeg,
        fizetesdatum,
        statusz,
        fizetesmod,
        megjegyzes,
        req.params.id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Fizetés módosítva!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {

    if(conn) conn.release();

  }

});
```

```
    }  
  });  
  
module.exports = router;
```

5. fizetesektorlelese.js

Megvalósítottam a fizetések törlésének funkcióját is. A törlés előtt megerősítő ablakot jelenítettem meg, hogy elkerüljem a véletlen adatvesztést. A rekord eltávolítása DELETE kérés segítségével történik. A backend sikeres válasza után értesítést jelenítettem meg a felhasználónak. A törlés befejezése után újra lekérdeztem az adatokat, hogy a lista naprakész maradjon.

```
const express = require("express");  
const router = express.Router();  
  
const pool = require("../db");  
  
router.delete("/:id", async (req, res) => {  
  let conn;  
  
  try {  
    conn = await pool.getConnection();  
  
    await conn.query(  
      "DELETE FROM fizetesek WHERE id = ?",  
      [req.params.id]  
    );  
  
    res.json({  
      siker: true,  
      uzenet: "Fizetés törölve!"  
    });  
  
  } catch (err) {  
    res.status(500).json(err);  
  
  } finally {  
    if(conn) conn.release();  
  
  }  
}
```



```
});  
  
module.exports = router;
```

fizetesek.css

Elkészítettem a fizetések oldalhoz tartozó CSS stíluslapot, amely biztosítja a felület egységes és modern megjelenését. A fő konténert úgy alakítottam ki, hogy a tartalom középre igazítva, megfelelő margókkal és térközökkel jelenjen meg. Az adatbeviteli űrlapot rácsszerkezet segítségével készítettem el, így különböző képernyőméreteken is jól használható marad. A gombokhoz és mezőkhöz lekerekített sarkokat, valamint lebegéskor megjelenő színváltást alkalmaztam a jobb felhasználói élmény érdekében.

```
.tenant-container {  
  padding: 30px;  
  max-width: 1400px;  
  margin: auto;  
}  
  
.tenant-container h1 {  
  text-align: center;  
  margin-bottom: 25px;  
}  
  
.tenant-form {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 15px;  
  margin-bottom: 30px;  
}  
  
.tenant-form input,  
.tenant-form select {  
  padding: 10px;  
  border: 1px solid #ccc;  
  border-radius: 8px;  
  font-size: 14px;  
}  
  
.tenant-form button {  
  background: #2563eb;  
  color: white;  
  border: none;  
  border-radius: 8px;  
  padding: 12px;  
  cursor: pointer;  
  transition: 0.3s;  
}  
  
.tenant-form button:hover {
```

```
    background: #1d4ed8;
}

.filter-container {
    margin-bottom: 25px;
}

.filter-container select {
    padding: 10px;
    border-radius: 8px;
    border: 1px solid #ccc;
}

.tenant-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
    gap: 20px;
}

.tenant-card {
    background: white;
    border-radius: 12px;
    padding: 20px;
    box-shadow: 0px 2px 10px rgba(0,0,0,0.1);
}

.tenant-card h3 {
    margin-bottom: 15px;
    color: #2563eb;
}

.tenant-card p {
    margin: 8px 0;
}

.buttons {
    display: flex;
    gap: 10px;
    margin-top: 15px;
}

.edit-btn {
    flex: 1;
    background: #f59e0b;
    color: white;
    border: none;
    padding: 10px;
    border-radius: 8px;
```

```
        cursor: pointer;
    }

    .edit-btn:hover {
        background: #d97706;
    }

    .delete-btn {
        flex: 1;
        background: #dc2626;
        color: white;
        border: none;
        padding: 10px;
        border-radius: 8px;
        cursor: pointer;
    }

    .delete-btn:hover {
        background: #b91c1c;
    }

    .active-status {
        color: green;
        font-weight: bold;
    }

    .expired-status {
        color: red;
        font-weight: bold;
    }

    @media (max-width: 768px) {

        .tenant-form {
            grid-template-columns: 1fr;
        }

        .tenant-grid {
            grid-template-columns: 1fr;
        }

    }
```

Szakmai Gyakorlat II. – Frontend – Backend integrációs szakasz
Orosz Kristóf – EYZWG9

localhost:3000/fizetesek

Fizetések

Válassz szerződést ▾

Összeg

éééé . hh . nn .

Teljesítve ▾

Fizetési mód

Megjegyzés

Fizetés rögzítése

Összes fizetés ▾

Bíró Zoltán

Összeg: 14500.00 Ft

Dátum: 2026. 06. 19.

Fizetési mód: Bankkártya

Megjegyzés: Új fizetési kötelezettség!

Státusz: Független

Módosítás

Törölés

1.Kép: Fizetések HTML oldala a funkciókkal ellátva.